ECHO to Veritas - Master Handoff

Updated: 2026-11-11. Local branch: `main`, based on `9d761e6`. FreeLLMAPI is ECHO's verified primary conversational provider; deterministic commands, security gates and offline fallback are preserved; optional free browser speech is implemented. Phase 1 (conversational follow-up navigation), Phase 4 (Lost & Found), Phase 5 (attendance, assignments, contacts), Phase 6 (timetable, announcements) and Phase 7 (model-assisted tool routing) are complete and verified end to end.FreeLLMAPI was re-verified
live on 2026-11-11, including real structured tool selection by the live model.
The model chooses which read-only tools answer a multi-module question, but every answer is still composed server-side from the authorized snapshot: no record text reaches the provider.

Phase 1 - conversational follow-up navigation (2026-11-09)

The reported defect: after ECHO explained a page and offered to open it, an acceptance such as "okay take me there" only produced more prose. Returning text is not navigation, so the flow was rebuilt around a structured action.

Implemented:

1. **Structured offer lifecycle (server).** An offer is stored per ECHO session with its action ID, path and label, bound to the account `sub` and the role, and expiring after 60 s. `server/services/echo.ts` owns the store; revocation (logout / New chat) deletes it.
2. **Deterministic acceptance before the provider.** Cancellation, explicit replacement ("actually, open assignments instead") and deictic acceptance ("take me there", "open it", "yes please", "show me", "go there", "let's do that", "alright") are resolved before any FreeLLMAPI call, so the primary brain cannot answer them as fresh conversation.
3. **Only offered pages become pending.** The pending action is registered only when the displayed reply names exactly one allowlisted page (or uses the canonical "say open X and I'll take you there" offer). Two destinations yield a clarification, never a guess. Offers can only come from the fixed topic allowlist, never from model-generated URLs.
4. **Verification, not claims.** The `done` SSE event now carries `action: { type: 'navigate', actionId, path, label }`. The widget forwards it via `echo-widget:action`; `EchoAssistant` re-validates the path against `ECHO_PAGES` and the current role, moves the real router, and confirms only after both the React location and `window.location` match. Failures say so.
5. **Client deferral.** `EchoCommandSession` now exposes `hasPending`, and bare follow-ups bypass local recognition (which previously answered "could not match") whenever no local confirmation is active.
6. **Invalidation.** Offers expire after 60 s, are cleared by cancellation, by a different destination, by executing, by an unrelated reply, by a role change, by a second account, and by logout/reset.

Phase 2 - expandable and full-screen workspace (2026-11-09)

The widget panel now has three presentation modes on one mounted conversation: compact (380 px), expanded (min(720 px, 92vw)) and full screen (100vw x 100dvh

with a centred readable column). The header gains a size button that cycles compact -> expanded -> full -> compact, with `aria-label` updated per mode ("Expand chat", "Full screen", "Restore size"). Escape restores compact before closing. The mode persists in `sessionStorage` (`echo-widget-mode-v1`) but the conversation, draft text, speech settings and pending server offers are never unmounted or reset by a mode change.

Verified in the browser regression at 1280x850: the panel widens on expand, fills the viewport in full screen, returns to compact, the URL never changes, and the message count is identical before and after each switch.

Phase 3 - optional voice input and the "Echo" wake word (2026-11-09)

Voice input is opt-in and free: no API key, no paid service, and no audio is sent to the ECHO server. The widget uses the browser's own `SpeechRecognition` (`webkitSpeechRecognition`) engine behind one small provider interface, so a host page can substitute another engine with `window.EchoWidgetRecognitionProviders` and `data-recognition-provider`.

- **Dictation.** The microphone button captures one utterance: interim text appears in the status line, and the final transcript fills the composer and is submitted through the same form path as typing. Portal commands, the FreeLLMAPI conversation, Stop and spoken replies therefore behave identically, and any draft the user already typed is preserved.
- **Wake word.** The "Echo" checkbox arms continuous recognition and acts only on utterances beginning with "Echo", "hey Echo", "okay Echo" or "hi Echo". Any other speech is ignored and never displayed, so room conversation cannot trigger navigation. The wake word is never restored across page loads: every reload needs a fresh opt-in.
- **Release points.** The recognizer is aborted while ECHO speaks (otherwise it transcribes its own voice) and re-armed when the speech queue drains, and it is released on Stop, New chat, logout/widget destroy, panel close and whenever the tab is hidden. Denied permission, a missing device, offline recognition and unsupported browsers produce explicit messages; text chat keeps working and the controls are hidden when no recognizer exists.

Limitations, stated plainly: the browser engine normally transcribes in the vendor's cloud, so audio leaves the device and recognition needs a network; it is missing in some browsers (notably Firefox) and needs microphone permission; and vendors suspend the stream in hidden tabs, so "Echo" is a foreground wake word only, not a background hotword. A true background wake word needs a native app or an always-on local model, neither of which exists here.

Verified with a simulated recognizer in `tests/echo-browser.mjs`: dictation start, interim text, spoken command navigation, one-shot stop, permission denial, Stop while listening, wake-word arming, ignoring speech without the wake word, wake-word command execution, no room-speech display, TTS pause/resume, New chat release, disarm, and a browser without any recognizer. These are simulated-API tests, **not** real microphone or audio verification; no real-device audio run was possible in this environment.

Workspace visualisations remain **not** implemented; Lost & Found retrieval is now implemented as Phase 4 below.

Phase 4 - authorized read-only data, Lost & Found (2026-11-11)

The reported conversation, reproduced and fixed:

> Can you read lost and found and answer me what all are actually found

It previously answered "Communication is for requesting a conversation with a teacher or coordinator...". Cause: `lost`/`found` were keywords of the communication topic, and no tool could reach records at all. Two defects were fixed - the topic mapping, so "open lost and found" no longer offers `/communication`, and the missing data path.

1. **A reader, not a prompt.** `createEchoServiceRouter({ readLostFound })` receives a callback that `server/index.ts` supplies only after the request is authenticated. It runs inside the same account-bound ECHO session, so no role claim or user id ever travels in the message text.
2. **Field allowlist.** `shapeLostFoundItems` in `server/services/echoData.ts` keeps id, type, item name, category, location, date, status and a truncated description. Reporter ids, contact info, claim notes and verification notes are dropped before display, memory or logging.
3. **Grounded answers.** The reply is composed on the server from those records: real counts, names, locations and claim status, plus explicit "nothing matches" and "records unavailable" paths. Lost & Found text is never sent to FreeLLMAPI, and history stores only a summary, so a later provider turn cannot receive the records either.
4. **Working memory.** The last result set is kept per ECHO session (30-minute TTL, deleted on revocation), so "the second one", "claim the blazer" and "which ones haven't been claimed" resolve without re-asking. A second session sees nothing.
5. **Cards.** `done.records` carries `{ id, title, subtitle, meta[], status }` and the widget renders each field with `textContent` only. A record containing markup or an instruction stays inert text.
6. **Stated limits.** Lost & Found is campus-wide and visible to any signed-in account, matching the existing `/api/lost-found` page. Attendance, marks and other per-student records are still not readable through ECHO.

Verified by 12 tests in `tests/echo-lostfound.test.ts`: the exact reported request, a plain list, repeat-turn variation, the unclaimed filter, an ordinal follow-up, a claim follow-up, a no-match search, a missing reader, cross-account isolation, field redaction, prompt-injection record content, and navigation still working. The browser regression also asserts real-database retrieval through the built-in engine, follow-up resolution, redaction in the raw SSE, card rendering and a hostile record that must not execute.

Phase 5 - application-wide module tools (2026-11-11)

The Lost & Found reader is now one instance of a shared layer rather than a special case. `server/services/echoTools.ts` owns shaping, intent detection, aggregation, phrasing and cross-module memory; `server/index.ts` owns scoping.

- **One snapshot, scoped server-side.** `readPortal(identity)` resolves the account id from the ECHO session, looks up that user's student row, and returns only what the existing page APIs would return: contacts (directory-wide), attendance and assignments for that student's own id, class and section. A staff account gets the directory and nothing else, because a teacher's view spans every student and per-record staff scoping is not designed.
- **Attendance** answers overall rate and per-subject rates from real rows, stating its arithmetic out loud (late counts as attended, excused sessions are excluded). "Which subject is lowest?" is computed, not assumed.
- **Assignments** reads real due dates and submission status: open work, due today/tomorrow/this week, overdue, and per-subject filters.
- **Contacts** searches the published directory by name, role, department and category, and falls back to academic staff for questions about submissions or marks instead of inventing a person.
- **Cross-module memory.** Analysing attendance stores the weakest subject, so "what assignments do I have in that subject?" resolves it. Memory is per ECHO session, 30-minute TTL, cleared on revocation, never shared across accounts.
- **Unavailable is stated.** If the host app cannot read a module, ECHO says so and emits no cards; it never estimates a percentage or a deadline.
- **Cards.** The same `done.records` channel renders module results, text-only.

Module coverage matrix

Module	Tool	Authorization	Status
Attendance	own sessions, per-subject rates	`students.userId = account id`	Implemented, tested
Assignments	own class/section work + own submissions, due/overdue filters	own class & section	Implemented, tested
Contacts	published staff directory	any signed-in account (mirrors `/api/contacts`)	Implemented, tested
Lost & Found	items and claim status	any signed-in account (mirrors `/api/lost-found`)	Implemented, tested
Examinations / marks	none	would need a provider-privacy decision	Not implemented
Timetable	none	read-only source exists (`/api/timetable`)	Not implemented
Announcements	none	read-only source exists (`/api/announcements`)	Not implemented
Leave, library, transport, portfolio	none	—	Not implemented
Staff attendance / assignments	none	per-record staff scoping undesigned	Not implemented

14 tests in `tests/echo-tools.test.ts` cover attendance arithmetic, weakest-subject calculation, cross-module subject follow-up, deadline and overdue filtering, staff search, the academic fallback, an unmatched search, an unreadable module, cross-account isolation, injection inside an assignment title, navigation still winning, and Lost & Found still working beside the new tools. Two intent boundaries were fixed during this work and are asserted by the existing suites: location questions ("where are my marks?") keep the navigation offer, and explanations ("tell me about assignments") are not answered with records.

Phase 6 - timetable and announcements (2026-11-11)

Both modules reuse the same contract: host-supplied snapshot, sanitized shape in `server/services/echoTools.ts`, deterministic answer, optional cards.

- **Timetable** reads only the signed-in student's own class and section (same filter as the page). It answers "what's my next class?" (next non-cancelled period after the current minute), a named weekday, today/tomorrow, which room a

subject uses, and lists periods with rooms, teachers and any substitution or cancellation. Staff accounts get an explicit "own class only" limitation.
- **Time and date handling.** The day is resolved with `Intl.DateTimeFormat` in `ECHO_PORTAL_TIMEZONE` (default `UTC`), never the host clock, and minute-of-day is computed in that zone. A weekend "today" points at the next school day instead of inventing periods. Times are displayed exactly as published; ECHO does not convert them. A regression test sets the zone to `America/Los_Angeles` with a UTC date that falls on the previous day in California and asserts the California weekday wins.
- **Announcements** mirror `GET /api/announcements` visibility exactly: approved notices, the account's own posts, or reviewer roles. Read state comes from that account's `announcement_reads` rows, so "any unread announcements?" is real. Keyword questions match title, content and category; emergency/critical priority is carried into the answer and the card.
- **Unchanged privacy.** Announcement and timetable text is data: it is rendered text-only and never sent to a provider.

Updated coverage matrix

Module	Tool	Authorization	Status
Attendance	own sessions, per-subject rates	`students.userId = account id`	Implemented, tested
Assignments	own class/section work + submissions, due/overdue filters	own class & section	Implemented, tested
Contacts	published staff directory	any signed-in account	Implemented, tested
Lost & Found	items and claim status	any signed-in account	Implemented, tested
Timetable	own class periods, next class, rooms, substitutions	own class & section	Implemented, tested
Announcements	visible notices, own unread state	mirrors `/api/announcements`	Implemented, tested
Examinations / marks	none	needs a provider-privacy decision	Not implemented
Leave, library, transport, portfolio	none	—	Not implemented
Staff attendance / assignments / timetable	none	per-record staff scoping undesigned	Not implemented

23 tests now live in `tests/echo-tools.test.ts`; the new ones cover next-class resolution against a fixed clock, a named day, weekend fallback, time-zone correctness, room lookup, announcement listing, unread filtering, keyword search, an unreadable source, and no cross-account timetable exposure.

Phase 7 - model-assisted tool routing (2026-11-11)

`server/services/echoToolSelect.ts` adds a router step that never becomes a data path. It is attempted only for **multi-module** messages (two or more module cues), so single-module questions and commands keep the deterministic fast path and add no provider latency.

The model sees the user message, the last four turns and a fixed catalogue of six tool names with descriptions. It returns only `{"tool_calls":[{"tool":"get_attendance","args":{"focus":"..."}}]}`.

Server-side guarantees, each covered by a test:

- **Allowlist.** Names outside the six tools are rejected and the reply says so without retrieving anything — an unknown tool can never execute.

- **Argument allowlist.** The only accepted key is `focus`, a string of at most 80 characters. `studentId`, role, SQL, table names and query strings all fail validation and are reported as invalid arguments.
- **Identity is never model-supplied.** Execution uses `readPortal(identity)` / `readLostFound` exactly as the deterministic path does; the ECHO session's `sub` is the only source of who is asking.
- **No records in the prompt.** The provider request is asserted (in a test) to contain no assignment title, room, teacher, announcement or Lost & Found text — only the catalogue. Answers are composed from the snapshot after selection.
- **Bounded execution.** At most three tools, duplicates dropped by `tool|args`, one provider call per message, 8 s timeout, and any failure falls through to the deterministic or conversational path. There is no retry loop.
- **Multi-module composition.** Attendance + assignments answered in one reply, with deduplicated text and up to eight cards. Follow-ups still use session memory, and the modules remain independently authorized.

Browser evidence (`tests/echo-tools-browser.mjs`, `npm run test:echo:tools-browser`): real login, real widget, real server and real session against a **stubbed router model** — one multi-module question produces both modules' text and at least two cards without navigating, and `show Aarav's ...` is refused with no data.This is browser coverage of the wiring. Live routing is verified separately: see "Phase 8" below.

Phase 8 - live routing evidence and deployment audit (2026-11-11)

`tests/echo-freellm-provider.mjs` now exercises the live model as a router, not only as a chat model. It passed twice on 2026-11-11 against FreeLLMAPI with two different upstream models (`gemini-robotics-er-2-preview` via `google/...` and `qwen3.8-27b` via `groq/...`), covering:

- an explicit multi-module question answered with **both** attendance and assignments (the deterministic path answers one module) with SSE `meta` attributed to `freellmapi`;
- indirect wording ("missing morning classes ... work I owe") still producing both modules;
- an unauthorized-account request returning only the signed-in student's own attendance, never another person's marks;
- no API key anywhere in the stream.

What the live runs exposed, and the changes made because of it:

- the model sometimes answered in prose instead of JSON, so router calls request `response_format: {type:"json_object"}` and retry exactly once without it when a gateway rejects the field;
- a routed turn was recorded as `local-guidance`, so diagnostics now attribute the turn to the model that actually chose the tools;
- the model often selected only the module it recognised first, so the reply is completed deterministically with any module the request itself names, from the same authorized snapshot - completion is server-side, never model-trusted;
- activation now also fires for one module cue plus a conjunction in a real sentence (>= 25 characters), because indirect requests named one module word;

if the model then selects nothing, a deterministic multi-module answer is used.

Observed limit: one earlier live run failed because the model replied in prose to the indirect request. Provider behaviour is not deterministic, so the fallback - not the model - is what guarantees an answer.

Deployment audit (2026-11-11)

- `vercel.json` and `api/index.ts` exist and build the Express app for Vercel's function runtime.
- **The backend is not reliable on Vercel as-is.** On serverless the SQLite file is `os.tmpdir()/veritas-data` and uploads are `/tmp/veritas-uploads`: both are ephemeral and per-instance, so users, records and uploads reset on cold start and differ between instances. ECHO sessions live in process memory and are lost on redeploy or another instance. This is a durability problem, not a setting.
- Production FreeLLMAPI connectivity needs a public HTTPS gateway; a loopback gateway is unreachable from Vercel, and non-loopback HTTP is rejected by design.
- No deploy was attempted: doing so would require accepting ephemeral student data or exposing a private gateway. The correct next step is a durable database (hosted Postgres/Turso), a shared session store, object storage for uploads and a hosted provider endpoint.

Phase 9 - durable staging foundation (2026-11-11)

The deployment path was audited before any storage change, then split into what could be verified locally without credentials or student data.

What changed

- **ECHO conversation state is durable.** History, Lost & Found working memory and cross-module subject memory moved behind `EchoStateStore` (`server/services/echoStateStore.ts`). Tests use the in-memory implementation; the server injects the SQLite one (`echoRuntimeStore.ts`). Every read and write is keyed by the authenticated `sub`, so a leaked sid cannot read another account's rows. Navigation offers and rate limits stay in memory on purpose: they are short-lived safeguards, not records.
- **Uploads are private.** The public `express.static('/uploads')` mount is gone. `/uploads/:folder/:name` requires a session and allows only the uploader, staff, or a file the portal already links from an announcement, assignment or submission (`uploadAccess.ts`, `uploadGuard.ts`). An `uploads` table records the owner so the rule survives restarts; legacy unreferenced files are staff-only.
- **Durable paths are configurable.** `VERITAS_DB_PATH` pins the database file so a mounted volume works even when the app directory is replaced on deploy; `VERITAS_REQUIRE_DURABLE=1` fails fast on serverless when it is unset.
- **Backups are a command.** `npm run db:backup` writes a transactionally consistent `VACUUM INTO` copy and verifies `integrity_check`. Run it before any schema change or host migration.

Evidence

- `npm run test:echo`: **204/204 PASS**, including 6 persistence tests.
- `npm run test:upload-access`: PASS - anonymous 401, another student 403, owner 200, staff 200, legacy file 403, path traversal refused, synthetic files only.

- The same run proves **restart persistence**: the live server writes ECHO history to `veritas_school.db`, the process is stopped, and a new SQLite connection reads that history back.
- `npm run db:backup`: PASS with `integrity_check: ok`.

Architecture options and cost (all zero mandatory fees)

Option	Durable DB	Uploads	Provider	Trade-off
Free VM (e.g. Oracle Always Free) + block volume + Caddy	Yes	Local disk	Cloudflare Tunnel	Keeps the synchronous SQLite layer and every test intact; needs server administration and a real backup routine
Vercel + Turso/libSQL free tier + R2 free tier	Yes	Yes	HTTPS gateway	Requires converting `node:sqlite` to async libSQL: a data-layer refactor
Vercel + Neon/Supabase free tier	Yes	Yes	HTTPS gateway	Same async refactor plus Postgres schema and dialect work
Paid Node host with a persistent disk	Yes	Local disk	Cloudflare Tunnel	Simplest operationally, but a recurring fee, so not chosen

Recommendation under the no-fee constraint: a free VM with a block volume, which keeps the existing synchronous SQLite layer and all 204 tests unchanged. Turso or Neon become the better serverless answer once the async conversion is approved.

Provider connectivity

A laptop gateway cannot be a production dependency. Free options are a named Cloudflare Tunnel (free; needs a domain you control for a stable hostname) or Tailscale Funnel (free; laptop must stay online). Neither is always available, so the safe fallback stands: without a reachable HTTPS gateway ECHO answers with deterministic portal guidance and never claims a model answered. No credential or tunnel is stored in this repository.

Staging plan (synthetic data only)

1. Provision the durable host and a block volume mounted at `/srv/veritas`.
2. Set `VERITAS_DATA_DIR=/srv/veritas/data`, `VERITAS_DB_PATH=/srv/veritas/data/veritas_school.db`, `VERITAS_UPLOAD_DIR=/srv/veritas/uploads`, `NODE_ENV=production`.
3. Set `SESSION_SECRET` (32+ chars), `ECHO_SESSION_SECRET`, `PORTAL_BOOTSTRAP_PASSWORD` (16+ chars), `ECHO_PORTAL_TIMEZONE=America/Los_Angeles`, and keep `ECHO_ALLOW_PAID_PROVIDER=false`.
4. Seed synthetic demo records with `npm run db:seed`, then run `npm run db:backup` immediately to capture the clean baseline.
5. Terminate TLS in front of the Node server; serve HTTPS only.
6. Verify: sign-in per role, two-account isolation, one multi-module ECHO answer, a private upload (owner 200, other student 403), a restart, and a restore from the backup file.
7. Only then discuss real student data. Do not import production records into an unverified host.

Still blocked

- No hosted database or volume was provisioned here (no credentials, no paid service), so the only durable store proven is a local SQLite file on a persistent path.

- Uploads are durable only on storage the operator provides; no object-store
driver is implemented yet.
- `node:sqlite` is synchronous, so hosted Postgres/libSQL needs an async data layer.
- No deployment was made, so there is still no production SHA to verify.

What changed

1. **FreeLLMAPI is the primary brain.** Natural conversation goes to the
FreeLLMAPI OpenAI-compatible gateway. `auto:reliable` is the default model and
the gateway chooses the upstream route dynamically. OpenAI is ignored unless
`ECHO_ALLOW_PAID_PROVIDER=true`; there is no mandatory paid provider.
2. **Security hardening.** ECHO sessions require a signed-in portal identity,
are bound to the account `sub`, reject unapproved origins, use no shipped
fallback secret, validate token format/expiry strictly, allowlist only
`message` and `page`, bound all stores and rate-limit messages.
3. **Production security hardening (this release):**
- CORS origin allowlisting replaced reflected `origin: true` with explicit
allowlist (403 on foreign origins, same-origin/preflight supported).
- Session secrets enforced: `SESSION_SECRET` ≥32 chars in production;
`assertSessionSecretConfigured()` fails at startup if unset.
- `PORTAL_BOOTSTRAP_PASSWORD` required (≥16 chars) in production; seeds the
principal's password; sample accounts receive random unusable hashes.
- Demo accounts (`/api/auth/demo-accounts`) return 404 in production; generic
forgot-password response instead of password text.
- Production password policy: 12-char minimum for self-service changes;
temporary one-time passwords for admin-created students/staff and resets.
- Login page: dev-only demo account panel; production shows "Secure sign■in
— scrypt-hashed credentials and role■based access control."
- AppShell: dev-only instant role switcher hidden in production.
- StudentsStaffPage: shows temporary password in production enrollment/reset
toasts.
- `server/utils/portalMode.ts`: production-mode detection.
- `server/utils/originPolicy.ts`: new CORS origin allowlist utility.
- `tests/portal-security.test.ts`: 6 new release-gate security tests.
3. **Natural conversation.** Greetings, typos, follow-ups, page/role context,
empathy, jokes, sensitive refusals, provider status and honest no-memory
behavior are handled without inventing school records.
4. **Fuzzy commands.** Politeness, location questions, "pull up", "bring up",
"where is" and location-aware navigation resolve locally. Compound plans are
atomic and execute nothing if any part is unsupported. Ordinary prose with
"and"/"or" stays conversational.
5. **Speech.** Free browser `speechSynthesis`, on/off, Stop, 0.8-1.5x speed,
sentence streaming, no overlap, session-scoped settings and text fallback. A
replacement provider can be registered at `window.EchoWidgetSpeechProviders`.
6. **Provider diagnostics.** Authenticated `/api/echo/provider` plus SSE
`event: meta` reveal provider/model/route without keys, prompts or replies.

Verified provider evidence

Command: `ECHO_USE_DESKTOP_SETTINGS=true node tests/echo-freellm-provider.mjs`.

- Re-verified 2026-11-11: real local gateway, **305 models visible**.
- Real multi-turn conversation; synthetic code `QX-7319` recalled on turn 2.
- Provider `freellmapi`, model `command-a-plus-05-2026`, route
`cohere/command-a-plus-05-2026`; turns **1192 ms and 1078 ms**.
- The live browser widget received the model-generated marker `PLUMAGE`.
- The gateway port `127.0.0.1:31415` was also confirmed open in this environment.
ECHO itself reports `configured: false` unless the desktop credentials are
opted into, which is why the model test passes `ECHO_USE_DESKTOP_SETTINGS=true`.
A port being open is not evidence by itself; the passing model test is.
- An earlier run routed to `qwen/qwen3.8-27b` via `groq`, confirming dynamic
routing. Do not hardcode an upstream model in tests.
- Credentials were read from the existing FreeLLMAPI desktop store only with
explicit opt-in, passed to the child server environment, never printed or
written to the repository or browser.

Verified local checks

Check	Result
204 unit/integration tests: config, commands, conversation, navigation, Lost & Found, module tools, tool routing, persistence, security	PASS
upload access e2e (`npm run test:upload-access`)	PASS (401/403/200/200, traversal refused, restart persistence)
tool-routing browser regression	PASS (stubbed router model; real server/session/widget)
live model routing (`ECHO_USE_DESKTOP_SETTINGS=true node tests/echo-freellm-provider.mjs`)	PASS twice on 2026-11-11 (two upstream models)
`tsc --noEmit`	PASS
production `vite build`	PASS
full browser regression	PASS
live FreeLLMAPI provider test	PASS (previous release; not rerun here)

`tests/echo-navigation.test.ts` (new) drives the real router and asserts the
`done.action` lifecycle: offer-then-accept, `open it`, `yes please`, `show me`,
no-pending ambiguity, cancellation, 60 s expiry, explicit replacement, role
change, session isolation, logout revocation, cross-account rejection and
provider bypass. `tests/echo-browser.mjs` adds a real built-in-engine browser
run (real widget, cookie session, real router) proving the URL changes from
`/assignments` to `/attendance` only after acceptance, plus a forced
`history.pushState` mismatch that must report "could not confirm".

Browser coverage includes login, typo commands, verified navigation feedback,
context help, clipboard success/failure, Stop/reset/error recovery, speech
on/off/speed/Stop/overlap/failure fallback, voice input (dictation, permission
denial, Stop while listening, wake word, TTS pause/resume, unsupported browser),
tool record cards, display modes, 390 px mobile, logout abort and second-account
isolation. It also drives the real built-in engine against the seeded database:
the reported Lost & Found question returns real items, "the second one" resolves,
and the raw SSE contains no reporter or contact fields. Browser regression uses
fake SSE for command flows and a simulated recognizer; only the opt-in test
proves a real model, and no test exercises a real microphone.

Configuration contract

Server-only variables (never `VITE_*`, never frontend code):

```text
FREELLMAPI_BASE_URL=https://<public-https-gateway>/v1
FREELLMAPI_API_KEY=<secret>
FREELLMAPI_MODEL=auto:reliable
ECHO_ALLOW_PAID_PROVIDER=false
ECHO_PORTAL_TIMEZONE=America/Los_Angeles
ECHO_SESSION_SECRET=<long random value>
SESSION_SECRET=<long random value>
PORTAL_BOOTSTRAP_PASSWORD=<16+ char production bootstrap secret>
```

- Development: with a key and no base URL, ECHO uses
`http://127.0.0.1:31415/v1`; the FreeLLMAPI desktop app must be running.
- Production: explicit HTTPS only. Credentials in URLs, query strings,
non-loopback HTTP and paths other than `/v1` are rejected.
- `PORTAL_BOOTSTRAP_PASSWORD` is required (≥16 chars) when `NODE_ENV=production`
or running on Vercel; seeds the principal's password; sample accounts get random
unusable hashes.
- Legacy `FREELMAPI_*` and `ECHO_MODEL_*` aliases still work.
- `ECHO_PORTAL_TIMEZONE` decides "today", "next class" and "due today". The
application stores no time-zone setting in its schema or school settings, so
there is no authoritative application value to read; the only authoritative
signal is the school record's California address. `.env.example` therefore sets
`America/Los_Angeles`. The default stays `UTC` for safety, and **`Asia/Kolkata`
was not applied**: it would come from the developer's location, not the school's
configuration. Override per deployment only with school confirmation.
- `/api/echo/provider` reports configured/reachable/model/base host; it never
exposes the key.

Vercel status and required action

No deployment was made in this phase. The workspace has no `.vercel` project
link and no Vercel CLI, so no production SHA exists to verify. `vercel.json`
(Vite build, `dist`, `api/index.ts`, `maxDuration: 30`) and `api/index.ts` are
present, so the app can be packaged for Vercel - but the current backend is not
reliable on serverless (see the deployment audit in Phase 8): SQLite lives in
`os.tmpdir()/veritas-data`, uploads in `/tmp/veritas-uploads`, and ECHO sessions
in process memory, all of which are ephemeral per instance.

To run FreeLLMAPI in production (after the durability work below):

1. Expose the gateway at a public HTTPS origin (an HTTPS tunnel or a hosted
FreeLLMAPI-compatible gateway).
2. Add `FREELLMAPI_BASE_URL`, `FREELLMAPI_API_KEY`, `FREELLMAPI_MODEL`,
`ECHO_SESSION_SECRET` and `SESSION_SECRET` in Vercel Project Settings ->
Environment Variables.
3. Redeploy and sign in.
4. Check `/api/echo/provider` for `configured: true`, `reachable: true`.
5. Send a model-only prompt and confirm the SSE `meta` provider/model.
6. If the values are absent, the deployment intentionally uses deterministic

fallback. HTTP 200 or Ready alone is not provider proof.

No credentials were exposed to frontend code. No paid provider is enabled.

Security status

`tests/echo-security.test.ts`: 135/135 PASS (including 6 portal-security gate tests). Fixed issues: anonymous issuance, unapproved origins, usable public fallback secret, malformed expiry, token suffix, unsupported fields, account binding, page-context injection, unbounded state and rate limiting.

New in this release:

- CORS origin allowlisting replaced reflected `origin: true` with explicit allowlist (403 on foreign origins, same-origin/preflight supported).
- Session secrets: `SESSION_SECRET` must contain at least 32 characters in production; `assertSessionSecretConfigured()` fails at startup if unset. `ECHO_SESSION_SECRET` added as alias.
- `PORTAL_BOOTSTRAP_PASSWORD` is required in production (`≥16 chars`); seeds random unusable hashes for sample accounts; only the bootstrap principal logs in.
- Demo accounts (`/api/auth/demo-accounts`) return 404 in production; generic forgot-password response instead of password text.
- Production password policy: 12-character minimum for self-service changes; temporary one-time passwords for admin-created students/staff and password resets.
- Login page: dev-only demo account panel; production shows "Secure sign■in — scrypt-hashed credentials and role■based access control."
- AppShell: dev-only instant role switcher hidden in production.
- StudentsStaffPage: shows temporary password in production enrollment/reset toasts.
- `server/utils/portalMode.ts`: production-mode detection (`NODE_ENV=production` or `VERCEL`).
- `server/utils/originPolicy.ts`: new CORS origin allowlist utility.
- `tests/portal-security.test.ts`: 6 new release-gate security tests.

Residual, pre-existing or architectural:

- `server/utils/auth.ts` still uses a public default cookie secret when `SESSION_SECRET` is unset. Set it privately in production; it is the outer gate.
- Portal-wide reflected CORS remains at the API gateway level; ECHO origin checks are an additional gate, not a replacement.
- History/revocation are process-local; serverless instances do not share them.
- Stop cannot guarantee upstream cancellation/billing.
- Route confirmation covers view state, not lazy page-data loading.
- No physical-device or multi-browser matrix yet.

Key files

- `server/services/echo.ts`: authenticated router, account-bound sessions, origin/body/rate checks, SSE, page context.
- `server/services/echoConversation.ts`: natural conversation, provider priority, FreeLLMAPI/OpenAI config, status/probe, usage metadata, topic allowlist (including the separate Lost & Found topic).
- `server/services/echoData.ts`: authorized read-only Lost & Found tool, field allowlist, intent detection, grounded answer composition, record cards.
- `server/services/echoTools.ts`: shared module tools (attendance, assignments,

contacts, timetable, announcements), zone-aware date handling, intent boundaries, aggregation, cross-module memory.
- `tests/echo-tools.test.ts`: 23 module-tool, memory and authorization tests.
- `server/services/echoToolSelect.ts`: model-assisted router, allowlist and argument validation, bounded execution, server-side composition.
- `tests/echo-tool-select.test.ts`: 9 routing, allowlist, no-record-leak and fallback tests.
- `tests/echo-tools-browser.mjs`: browser regression for multi-module answers and refusal (stubbed router model).
- `tests/echo-lostfound.test.ts`: 12 Lost & Found retrieval, memory and authorization tests.
- `server/index.ts`: authenticated `/api/echo/config`, `/api/echo/provider`, ECHO mount with cookie/Bearer portal identity.
- `src/lib/echoCommands.ts`, `src/lib/echoRecognition.ts`: registry, fuzzy commands, atomic plans, prose guards.
- `echo-web/widget/widget.js`: speech controller, credentials, page context, Stop/reset/copy.
- `tests/echo-navigation.test.ts`: 14 Phase 1 navigation lifecycle tests.
- `src/components/EchoAssistant.tsx`: `echo-widget:action` handler (allowlist, role check, router execution, observed-route confirmation).
- `tests/echo-security.test.ts`: 12 release gates.
- `tests/portal-security.test.ts`: 6 new release-gate security tests.
- `tests/echo-conversation.test.ts`: 26 conversational/provider tests.
- `tests/echo-commands.test.ts`: 85 command tests.
- `tests/echo-browser.mjs`: full browser regression including speech.
- `tests/echo-freellm-provider.mjs`: opt-in real-provider verification.
- `server/utils/originPolicy.ts`: new CORS origin allowlist utility.
- `server/utils/portalMode.ts`: production-mode detection.
- `.env.example`, `README.md`, `echo-web/README.md`: configuration contract.

Git and publication state

Previous release commit: `a49b9db8cacbb9c81490da4b209931ea45a1fa7`.
Base for this work: `9d761e6`, rebased onto upstream `5cfd8da` (feat(portal): support PORTAL_DEMO_MODE) before pushing, so history is linear.
Phase 1 navigation: `0013123` (feat(echo): execute offered navigation from conversational follow-ups). Phase 2 workspace modes: `6cef2ab` (feat(echo): add expanded and full-screen workspace modes). Phase 3 voice input: `89202cb` (feat(echo): add optional microphone input and a foreground "Echo" wake word). Phase 4 Lost & Found: `708fe80` (feat(echo): answer Lost & Found questions from authorized records). Phase 5 module tools: `4a99067` (feat(echo): generalize authorized read-only tools across modules). Phase 6 timetable and announcements: `ad0e592` (feat(echo): read timetable and announcements with zone-aware dates). Phase 7 tool routing: `664fd44`; Phase 8 live routing, fallback completion and provider attribution: `67a0e81`; Phase 9 durable ECHO state, private uploads and backups: `c33ee53`. Documentation commits sit on top; use `git log --oneline` for the exact current HEAD.

`npm run test:echo`: **204/204 PASS**.
`tsc --noEmit`: PASS.
`npm run build`: PASS (2045 modules).

`npm run test:echo:browser`: PASS, including the real built-in engine run.

No deployment exists to compare against: the workspace has no `.vercel` project link, no deploy hook and no Vercel CLI. Once durability work is done, set `SESSION_SECRET`, `ECHO_SESSION_SECRET`, `PORTAL_BOOTSTRAP_PASSWORD`, `FREELLMAPI_BASE_URL`, `FREELLMAPI_API_KEY`, `FREELLMAPI_MODEL`, `ECHO_PORTAL_TIMEZONE=America/Los_Angeles` and `ECHO_ALLOW_PAID_PROVIDER` in Vercel Project Settings -> Environment Variables, deploy, then verify `/api/echo/provider` reports `configured: true` and `reachable: true` and that a routed multi-module prompt answers with both modules on the deployed origin. HTTP 200 or Ready status alone is not provider proof. Do not use `VITE_*` variables and do not force-push.

Remaining priorities (not implemented)

1. **Remaining modules.** Examinations/marks, leave, library, transport and portfolio have no tool. Marks need a privacy decision first (nothing private is sent to a provider today; answers would be composed server-side). Staff rows stay out until per-record scoping is designed, because a teacher's view spans every student.
2. **Production hosting durability.** The serverless backend stores SQLite and uploads in ephemeral `/tmp` and keeps ECHO sessions in process memory, and the FreeLLMAPI gateway is loopback-only. Production needs a durable hosted database, a shared session store, object storage for uploads and a public HTTPS provider endpoint before a deploy can be called reliable. No deployment was made; no production SHA exists to verify.
3. **Charts.** No chart is rendered. Cards and text only; `recharts` is available and attendance/assignment data is now structured enough to chart once there is a validated schema.
2. **Charts/tables in the workspace.** `recharts` is already a dependency, but ECHO renders text only. Structured results need server-validated payloads first, and only where real supported data exists.
3. **File upload / multimodal input.** Not present.
4. **Real audio and real-provider verification.** Voice input is verified with a simulated recognizer only. A real microphone run needs a machine with a device and a foreground browser, which this environment does not provide.

Exact handoff paths

- Markdown: `F:\Downloads\Veritas-Echo-Integration\echo-web\docs\ECHO-WEB-MASTER-HANDOFF.md`
- PDF: `F:\Downloads\Veritas-Echo-Integration\echo-web\docs\ECHO-WEB-MASTER-HANDOFF.pdf`
- Backups: `F:\ECHO-Handoff\ECHO-WEB-MASTER-HANDOFF.md` and `.pdf`
- PDF generator: `python echo-web/docs/render_handoff.py` (requires reportlab; output text is selectable/copyable).

Ready-to-copy continuation prompt

Continue ECHO/Veritas at `F:\Downloads\Veritas-Echo-Integration` on local `main`. Read this handoff, `PENDING_WORK.md`, `RELEASE_VERIFICATION.md` and

`FUZZY_COMMANDS.md`, then inspect `git status`, `git log` and remotes. Phase 1 (conversational follow-up navigation with a verified, allowlisted route action) Phase 2 (compact/expanded/full-screen workspace) and Phase 3 (optional browser microphone dictation plus a foreground-only "Echo" wake word, wired to the same command/conversation/Stop/TTS path) are complete. FreeLLMAPI is still the primary conversational brain, with deterministic commands, security gates, offline fallback and free browser speech in both directions.

Phases 4-7 are complete: six data modules through `server/services/echoTools.ts` with cross-module subject memory, plus model-assisted routing (`server/services/echoToolSelect.ts`) that keeps every answer server-composed. FreeLLMAPI re-verified live on 2026-11-11 (305 models, `command-a-plus-05-2026`, 1.08-1.19 s turns). Next priorities, in order: (1) examinations only after a privacy decision; (2) charts for validated attendance and assignment data; (3) live-model routing evidence and per-record staff scoping. The portal has no documented grade-projection feature, so do not invent one. Real microphone and real-device audio verification still needs hardware this environment lacks.

Do not recreate completed features, weaken tests or enable a paid provider. After any change, rerun `npm run test:echo` (204 tests), `npx tsc --noEmit`, `npm run build` and `npm run test:echo:browser`; the opt-in provider test needs the FreeLLMAPI desktop gateway. Keep the Markdown/PDF handoffs and `F:\ECHO-Handoff` copies current. Verify the deployed site after pushing; do not claim provider success from HTTP 200 or Ready alone.